# جزوه درجه یک سیستم عامل

## فصل ۳ - بخش ۴: ارتباط بین پردازه‌ای (IPC)

در بخش قبل دیدیم که چطور پردازه‌ها به دنیا می‌آیند و از دنیا می‌روند. اما پردازه‌ها در یک جزیره خالی زندگی نمی‌کنند. آن‌ها نیاز دارند با هم "حرف" بزنند.

### *الف) پردازه مستقل در مقابل پردازه همکار*

- **پردازه مستقل (Independent):** یک پردازه منزوی و گوشه‌گیر است. روی هیچ پردازه دیگری تأثیر نمی‌گذارد و از هیچ پردازه‌ای تأثیر نمی‌پذیرد. (مثلاً یک ویرایشگر متن ساده که فقط روی فایل خودش کار می‌کند).
- **پردازه همکار (Cooperating):** پردازه‌ای است که با بقیه پردازه‌ها داده مشترک دارد، روی آن‌ها تأثیر می‌گذارد یا از آن‌ها تأثیر می‌پذیرد. (مثلاً مرورگر وب شما که با یک سرور وب در ارتباط است).

**چرا به همکاری نیاز داریم؟ (سوال امتحانی)**

1. **اشتراک اطلاعات:** چند پردازه می‌خواهند به یک داده مشترک دسترسی داشته باشند (مثل قابلیت Copy و Paste که حافظه Clipboard بین چند برنامه مشترک است).
2. **تسریع محاسبات:** یک کار سنگین را به چند کار کوچکتر تقسیم می‌کنیم و هر کدام را به یک پردازه (یا هسته CPU) می‌سپاریم تا *موازی* انجام شوند و کار زودتر تمام شود.
3. **پیمانه‌سازی (Modularity):** به جای ساختن یک نرم‌افزار غول‌پیکر و یکپارچه، آن را به بخش‌های کوچکتر و مستقل (پیمانه) تقسیم می‌کنیم. هر بخش یک پردازه جدا است. (مثل معماری مرورگر کروم).

**مثال: معماری مرورگر کروم** کروم یک نمونه عالی از پیمانه‌سازی است. به جای اینکه کل مرورگر یک پردازه باشد، کروم از چند نوع پردازه استفاده می‌کند:

- **پردازه Browser:** مسئول دکمه‌ها، نوار آدرس و مدیریت کلی (فقط یکی از آن وجود دارد).
- **پردازه Renderer (پردازشگر):** مسئول نمایش محتوای وب‌سایت (HTML, JavaScript). **هر تب (Tab) یک پردازه Renderer جداگانه است.**
- **پردازه Plug-in:** برای هر پلاگین مثل Flash یک پردازه جدا ساخته می‌شود.

**مزیت این کار چیست؟** اگر یک وب‌سایت در یک تب هنگ کند و پردازه Renderer آن از کار بیفتد، **فقط همان تب از کار می‌افتد** و بقیه تب‌ها و خود مرورگر به کارشان ادامه می‌دهند. به این کار "جداسازی" یا **Sandboxing (گودال شنی)** هم می‌گویند که امنیت را بسیار بالا می‌برد.

### *ب) دو مدل اصلی برای ارتباط (IPC)*

برای اینکه دو پردازه با هم صحبت کنند، دو راه اصلی وجود دارد:

**۱. مدل حافظه مشترک (Shared Memory):**

- **ایده:** سیستم عامل یک "اتاق" یا "تخته سیاه" (یک قطعه حافظه) را در اختیار دو پردازه قرار می‌دهد. هر دو پردازه می‌توانند روی این تخته سیاه بنویسند و از روی آن بخوانند.

- **سرعت: بسیار سریع.** بعد از اینکه سیستم عامل آن "اتاق" را ساخت (که فقط یک بار نیاز به دخالت سیستم عامل یا System Call دارد)، پردازه‌ها می‌توانند با سرعت نور و بدون هیچ دخالت اضافه‌ای از کرنل، با هم داده رد و بدل کنند.
- **چالش:** مدیریت این تخته سیاه **با خودِ برنامه‌نویس است**. اگر هر دو پردازه همزمان بخواهند یک جا بنویسند، همه‌چیز به هم می‌ریزد (مشکل همگام‌سازی).

**۲. مدل عبور پیغام (Message Passing):**

- **ایده:** پردازه‌ها مستقیماً با هم در ارتباط نیستند. آن‌ها از طریق "اداره پست" (یعنی **هسته سیستم عامل** یا Kernel) برای هم "نامه" (پیغام) می‌فرستند.
- **عملیات:** `send(پیغام)` و `receive(پیغام)`.
- **سرعت: کندتر است.** چون هر ارسال و هر دریافت یک نامه، نیازمند دخالت سیستم عامل (System Call) است.
- **مزیت: بسیار ساده‌تر و امن‌تر.** سیستم عامل خودش مدیریت صف نامه‌ها و نوبت‌دهی را انجام می‌دهد. نیازی نیست نگران باشیم که دو پردازه همزمان به یک داده دسترسی پیدا کنند. این مدل برای **سیستم‌های توزیع‌شده** (ارتباط بین دو کامپیوتر مختلف) عالی است.

# فصل ۳ - بخش ۵: IPC در سیستم‌های حافظه مشترک

بیایید روی مدل اول (حافظه مشترک) عمیق شویم.

### *الف) مسئله تولیدکننده - مصرف‌کننده (Producer-Consumer)*

این یک الگوی کلاسیک در مهندسی کامپیوتر است که با حافظه مشترک حل می‌شود.

- **تولیدکننده (Producer):** پردازه‌ای که داده "تولید" می‌کند (مثلاً یک سرور وب که فایل HTML می‌سازد).
- **مصرف‌کننده (Consumer):** پردازه‌ای که داده "مصرف" می‌کند (مثلاً مرورگر شما که آن فایل را نمایش می‌دهد).

هر دو باید به یک "بافر" (Buffer) مشترک دسترسی داشته باشند. تولیدکننده داده را در بافر می‌گذارد و مصرف‌کننده آن را از بافر برمی‌دارد.

- **بافر نامحدود (Unbounded):** اندازه بافر هیچ محدودیتی ندارد. تولیدکننده هرگز منتظر نمی‌ماند.
- **بافر محدود (Bounded):** اندازه بافر ثابت است (مثلاً ۱۰ خانه).
  - اگر بافر **پر** باشد، تولیدکننده باید **منتظر** بماند تا جا خالی شود.
  - اگر بافر **خالی** باشد، مصرف‌کننده باید **منتظر** بماند تا داده‌ای تولید شود.

### *ب) پیاده‌سازی بافر محدود (خیلی مهم)*

ما از یک "آرایه چرخشی" (Circular Array) به عنوان بافر استفاده می‌کنیم. برای مدیریت این آرایه، به دو "اشاره‌گر" یا اندیس نیاز داریم که آن‌ها هم در حافظه مشترک قرار دارند:

- `in`: اندیس خانه‌ای که تولیدکننده **بعداً** در آن خواهد نوشت (محل خالی بعدی).
- `out`: اندیس خانه‌ای که مصرف‌کننده **بعداً** از آن خواهد خواند (اولین خانه پُر).

**دو حالت کلیدی:**

1. **بافر خالی است:** `in == out`
   - مصرف‌کننده باید منتظر بماند (در یک حلقه `while (in == out)` گیر می‌کند).
1. **بافر پُر است:** `(in + 1) % BUFFER_SIZE == out`
   - تولیدکننده باید منتظر بماند (در یک حلقه `while ((in + 1) % BUFFER_SIZE == out)` گیر می‌کند).

**نکته:** این کدهای `while` که فقط منتظر می‌مانند، "انتظار فعال" یا "Busy-Waiting" نام دارند و CPU را اشغال می‌کنند. در فصل‌های بعد (همگام‌سازی) یاد می‌گیریم چطور بدون اشغال CPU، پردازه را "بخوابانیم".

# فصل ۳ - بخش ۶: IPC در سیستم‌های عبور پیغام

حالا به مدل دوم (اداره پست) می‌پردازیم. اینجا دیگر حافظه مشترکی در کار نیست.

### الف) مفاهیم پایه: `send` و `receive`

برای ارسال و دریافت پیغام، چند سوال اساسی در طراحی سیستم عامل پیش می‌آید:

**۱. نام‌گذاری (چطور همدیگر را پیدا کنیم؟)**

- **ارتباط مستقیم (Direct):**
  - پردازه‌ها باید **اسم** همدیگر را به طور واضح بدانند.
  - `send(P, message)`: پیغام را به پردازه P بفرست.
  - `receive(Q, message)`: از پردازه Q پیغام بگیر.
  - **عیب:** انعطاف‌پذیری کم است. اگر اسم پردازه عوض شود، کل کد باید عوض شود (Hard-coding).
- **ارتباط غیرمستقیم (Indirect):**
  - پردازه‌ها از طریق یک واسطه به نام **صندوق پستی (Mailbox)** یا **پورت (Port)** با هم حرف می‌زنند.
  - هر دو پردازه یک صندوق پستی مشترک را می‌شناسند (مثلاً صندوق شماره A).
  - `send(A, message)`: پیغام را به صندوق A بفرست.
  - `receive(A, message)`: از صندوق A پیغام بگیر.
  - **مزیت:** انعطاف‌پذیری بالا. پردازه‌ها اصلاً لازم نیست همدیگر را بشناسند.
  - **مالکیت صندوق:** صندوق می‌تواند متعلق به یک پردازه باشد (که اگر پردازه بمیرد، صندوق هم از بین می‌رود) یا متعلق به سیستم عامل باشد (که مستقل است و امن‌تر است).

**۲. همگام‌سازی (مسدودشونده یا بی‌انسداد؟)**

این بخش به شدت مهم است و رفتار "ارسال" و "دریافت" را مشخص می‌کند.

- **ارسال مسدودشونده (Blocking Send):**
  - فرستنده پیغام را می‌فرستد و **مسدود می‌شود** (می‌ایستد) تا زمانی که گیرنده پیغام را تحویل بگیرد. (مثل زنگ زدن به کسی و منتظر ماندن تا گوشی را بردارد).
- **ارسال بی‌انسداد (Non-blocking Send):**
  - فرستنده پیغام را می‌فرستد و **بلافاصله** به ادامه کارش می‌رسد، چه گیرنده پیغام را گرفته باشد چه نه. (مثل فرستادن ایمیل یا SMS).
- **دریافت مسدودشونده (Blocking Receive):**

- گیرنده منتظر پیغام می‌ماند. اگر پیغامی نباشد، **مسدود می‌شود** (می‌ایستد) تا زمانی که یک پیغام جدید برسد. (رفتار پیش‌فرض).

- **دریافت بی‌انسداد (Non-blocking Receive):**
  - گیرنده چک می‌کند؛ اگر پیغامی بود، آن را برمی‌دارد. اگر نبود، یک پیغام "خالی" (Null) می‌گیرد و بلافاصله به کارش ادامه می‌دهد (منتظر نمی‌ماند).

**ترکیب طلایی (Rendezvous):** اگر هم `send` و هم `receive` هر دو **مسدودشونده** باشند، به آن **"ملاقات" (Rendezvous)** می‌گویند. فرستنده و گیرنده باید دقیقاً در یک لحظه به نقطه ارتباط برسند و تا زمانی که تبادل انجام نشود، هر دو منتظر می‌مانند.

**۳. بافرسازی (صف نامه‌ها چقدر است؟)**

پیغام‌ها قبل از دریافت در یک "صف" موقت منتظر می‌مانند.

- **ظرفیت صفر (Zero Capacity):**
  - هیچ صفی وجود ندارد. فرستنده باید **حتماً** منتظر بماند تا گیرنده آماده دریافت شود. (این همان **Rendezvous** است).
- **ظرفیت محدود (Bounded Capacity):**
  - صف به اندازه `n` پیغام جا دارد.
  - اگر صف پُر باشد، فرستنده **مسدود** می‌شود (می‌ایستد) تا جا خالی شود.
- **ظرفیت نامحدود (Unbounded Capacity):**
  - صف بی‌نهایت جا دارد.
  - فرستنده **هرگز** مسدود نمی‌شود (همیشه ارسال می‌کند و می‌رود).

**خلاصه نکات کلیدی این بخش:**

1. **دو مدل IPC:** حافظه مشترک (سریع، سخت) و عبور پیغام (کندتر، آسان).
2. **مسئله تولیدکننده-مصرف‌کننده:** یک الگوی کلاسیک برای حافظه مشترک با استفاده از بافر محدود (آرایه چرخشی با `in` و `out`).
3. **طراحی عبور پیغام:** مستقیم (نام بردن) یا غیرمستقیم (صندوق پستی).
4. **همگام‌سازی:** مسدودشونده (منتظر می‌ماند) یا بی‌انسداد (ادامه می‌دهد).
5. **بافرینگ:** ظرفیت صفر (ملاقات)، محدود (انتظار در صورت پُر بودن)، نامحدود (بدون انتظار).